is there any unified format for storing data from strength training? reps, series, superseries, weights

Short answer: **there is no single, universally adopted standard** for strength-training data.
Long answer: there *are* a few **de-facto patterns** that most apps, coaches, and data scientists converge on, depending on how structured you want the data to be.

Below is the practical landscape so you can choose wisely.

Why there's no true "standard"

Strength training is messy compared to endurance sports:

- Variable set structures (straight sets, pyramids, drop sets)
- Supersets / giant sets / circuits
- RPE, tempo, rest, pauses
- Changing loads within a set
- Different goals (hypertrophy vs powerlifting vs CrossFit)

That flexibility makes a single rigid format impractical.

The 3 common data models that *actually work*

1. Set-centric (most common, most flexible)

This is the closest thing to a de-facto standard.

Each **set** is a row/object.

Example

```
{
  "exercise": "Back Squat",
  "set_number": 3,
  "reps": 5,
```

```
  "weight": 140,
  "unit": "kg",
  "rpe": 8,
  "tempo": "31X1",
  "rest_sec": 180
}
```

How supersets are handled

```
"superset_id": "A1"
```

Used by

- Strong, Hevy, TrainerRoad strength beta
- Most coach spreadsheets
- Wearables that export strength data

Pros

- Handles *everything*
- Easy to query and analyze
- Scales well

Cons

- Verbose
- Slightly annoying for manual entry

👉 **If you want longevity and interoperability, this is the safest choice.**

2. Exercise-block (human-friendly, semi-structured)

This is what most people *write*, even if machines later normalize it.

Example

```
{
  "exercise": "Bench Press",
  "sets": [
    { "reps": 8, "weight": 80 },
    { "reps": 8, "weight": 80 },
    { "reps": 6, "weight": 85 }
  ]
}
```

Supersets

```
"superset": ["Bench Press", "Barbell Row"]
```

Pros

- Easy for humans
- Compact
- Mirrors training logs

Cons

- Harder to analyze at scale
- Superset timing/rest gets fuzzy

👉 Good for **apps and UX**, usually converted internally to set-centric.

3. Workout graph model (most expressive, least adopted)

This treats training as a **DAG (directed graph)**.

- Nodes = sets
- Edges = order, rest, or dependency
- Supersets are parallel branches

Pros

- Perfectly models real training
- Handles circuits, EMOMs, complexes

Cons

- Overkill for 95% of use cases
- Almost no consumer tools support it

👉 Used in research or advanced coaching systems only.

What about existing “standards”?

There are **attempts**, but none won:

- **FIT / FIT-SDK** → strength support is weak
- **Open mHealth** → too abstract
- **TrainingPeaks / Intervals.icu** → endurance-centric
- **GymML datasets** → research-focused, not practical

No equivalent of **GPX** for lifting exists.

My blunt recommendation

If you're designing **anything serious** (app, analysis pipeline, long-term log):

> **Store everything as individual sets with optional grouping IDs**

Minimum viable schema:

```
workout_id
timestamp
exercise_id
set_index
reps
load
unit
superset_id (nullable)
rpe (optional)
tempo (optional)
rest_sec (optional)
```

You can always:

- Render it nicely for humans
- Export it to text formats
- Extend it later without breaking history

If you want, tell me **what you're building** (personal log, app, research, AI coach), and I'll help you design a clean schema that won't paint you into a corner.

create a JSON schema for the bench-centric

Here's a **clean, practical JSON Schema** for an **exercise-/bench-centric (exercise-block) strength log**, where data is grouped by exercise and contains sets inside it.

This follows **JSON Schema Draft 2020-12** and is designed so you can later normalize it into set-centric form if needed.

Bench-centric workout schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/strength-workout-bench-centric.json"
  "title": "Strength Training Workout (Bench-centric)",
  "type": "object",
  "required": ["workout_id", "date", "exercises"],
  "properties": {
    "workout_id": {
      "type": "string",
      "description": "Unique workout identifier"
    },
    "date": {
      "type": "string",
      "format": "date-time"
    },
    "notes": {
      "type": "string"
    },
    "exercises": {
      "type": "array",
      "minItems": 1,
      "items": {
        "$ref": "#/$defs/exerciseBlock"
      }
    }
  },
  "$defs": {
    "exerciseBlock": {
      "type": "object",
      "required": ["name", "sets"],
      "properties": {
        "exercise_id": {
          "type": "string",
          "description": "Optional stable ID for the exercise"
        },
        "name": {
          "type": "string",
          "description": "Exercise name (e.g. Bench Press)"
        },
        "superset_id": {
          "type": "string",
          "description": "Exercises sharing the same ID are performed as a
        },
        "equipment": {
          "type": "string",
          "enum": ["barbell", "dumbbell", "machine", "bodyweight", "cable"
        },
        "sets": {
          "type": "array",
          "minItems": 1,
```

```
        "items": {
          "$ref": "#/$defs/set"
        }
      },
      "notes": {
        "type": "string"
      }
    }
  },
  "set": {
    "type": "object",
    "required": ["reps"],
    "properties": {
      "reps": {
        "type": "integer",
        "minimum": 0
      },
      "weight": {
        "type": "number",
        "minimum": 0,
        "description": "Load used (omit for bodyweight-only sets)"
      },
      "unit": {
        "type": "string",
        "enum": ["kg", "lb"]
      },
      "rpe": {
        "type": "number",
        "minimum": 1,
        "maximum": 10
      },
      "rir": {
        "type": "integer",
        "minimum": 0
      },
      "tempo": {
        "type": "string",
        "pattern": "^[0-9X]{3,4}$",
        "description": "E.g. 31X1"
      },
      "rest_sec": {
        "type": "integer",
        "minimum": 0
      },
      "completed": {
        "type": "boolean",
        "default": true
      }
    },
    "additionalProperties": false
  }
},
```

```
  "additionalProperties": false
}
```

Example valid instance

```
{
  "workout_id": "w_2026_01_23",
  "date": "2026-01-23T18:45:00Z",
  "exercises": [
    {
      "name": "Bench Press",
      "equipment": "barbell",
      "superset_id": "A",
      "sets": [
        { "reps": 8, "weight": 80, "unit": "kg", "rpe": 7 },
        { "reps": 8, "weight": 80, "unit": "kg", "rpe": 8 },
        { "reps": 6, "weight": 85, "unit": "kg", "rpe": 9 }
      ]
    },
    {
      "name": "Barbell Row",
      "equipment": "barbell",
      "superset_id": "A",
      "sets": [
        { "reps": 10, "weight": 70, "unit": "kg" }
      ]
    }
  ]
}
```

Design notes (why this works)

- **Human-friendly**: mirrors how lifters think and write logs
- **Supersets**: handled via shared `superset_id`
- **Future-proof**: easy to flatten into a set-centric table later
- **Extensible**: you can add velocity, bands, pauses, or fatigue metrics without breaking existing data

If you want, I can:

- Add **drop sets / pyramids**
- Add **time-based or AMRAP sets**
- Show the **exact transformation** to a set-centric schema

- Optimize this for **database storage or analytics**

Just say what the next step is.

create a JSON schema for the set-centric

Below is a **production-ready JSON Schema for a set-centric strength-training log**. This is the format coaches, analytics pipelines, and serious apps converge on.

It uses **JSON Schema Draft 2020-12**, is strict by default, and explicitly supports supersets, circuits, and future extensions.

Set-centric workout JSON Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/strength-workout-set-centric.json",
  "title": "Strength Training Workout (Set-centric)",
  "type": "object",
  "required": ["workout_id", "date", "sets"],
  "properties": {
    "workout_id": {
      "type": "string",
      "description": "Unique workout identifier"
    },
    "date": {
      "type": "string",
      "format": "date-time"
    },
    "athlete_id": {
      "type": "string"
    },
    "notes": {
      "type": "string"
    },
    "sets": {
      "type": "array",
      "minItems": 1,
      "items": {
        "$ref": "#/$defs/set"
      }
    }
  },
```

```
"additionalProperties": false,

"$defs": {
  "set": {
    "type": "object",
    "required": [
      "set_id",
      "exercise",
      "set_index",
      "reps"
    ],
    "properties": {
      "set_id": {
        "type": "string",
        "description": "Globally unique set identifier"
      },
      "exercise": {
        "type": "object",
        "required": ["name"],
        "properties": {
          "exercise_id": {
            "type": "string"
          },
          "name": {
            "type": "string",
            "description": "Exercise name (e.g. Bench Press)"
          },
          "equipment": {
            "type": "string",
            "enum": [
              "barbell",
              "dumbbell",
              "machine",
              "bodyweight",
              "cable",
              "kettlebell",
              "other"
            ]
          }
        },
        "additionalProperties": false
      },
      "set_index": {
        "type": "integer",
        "minimum": 1,
        "description": "Order of the set for this exercise"
      },
      "reps": {
        "type": "integer",
        "minimum": 0
      },
      "weight": {
```

```
          "type": "number",
          "minimum": 0
        },
        "unit": {
          "type": "string",
          "enum": ["kg", "lb"]
        },
        "rpe": {
          "type": "number",
          "minimum": 1,
          "maximum": 10
        },
        "rir": {
          "type": "integer",
          "minimum": 0
        },
        "tempo": {
          "type": "string",
          "pattern": "^[0-9X]{3,4}$",
          "description": "E.g. 31X1"
        },
        "rest_sec": {
          "type": "integer",
          "minimum": 0
        },
        "superset_id": {
          "type": "string",
          "description": "Sets with the same ID belong to the same superse
        },
        "block_id": {
          "type": "string",
          "description": "Optional higher-level grouping (e.g. circuit, EM
        },
        "completed": {
          "type": "boolean",
          "default": true
        },
        "timestamp": {
          "type": "string",
          "format": "date-time",
          "description": "Optional precise completion time"
        },
        "notes": {
          "type": "string"
        }
      },
      "additionalProperties": false
    }
  }
}
```

Example valid instance

```
{
  "workout_id": "w_2026_01_23",
  "date": "2026-01-23T18:45:00Z",
  "sets": [
    {
      "set_id": "s1",
      "exercise": { "name": "Bench Press", "equipment": "barbell" },
      "set_index": 1,
      "reps": 8,
      "weight": 80,
      "unit": "kg",
      "rpe": 7,
      "superset_id": "A"
    },
    {
      "set_id": "s2",
      "exercise": { "name": "Barbell Row", "equipment": "barbell" },
      "set_index": 1,
      "reps": 10,
      "weight": 70,
      "unit": "kg",
      "superset_id": "A"
    },
    {
      "set_id": "s3",
      "exercise": { "name": "Bench Press", "equipment": "barbell" },
      "set_index": 2,
      "reps": 6,
      "weight": 85,
      "unit": "kg",
      "rpe": 9
    }
  ]
}
```

Why this schema is solid

- **One row = one set** → perfect for databases and analytics
- **Supersets & circuits** handled cleanly via IDs
- **Order preserved** via `set_index` (exercise-local) and optional `timestamp`
- **Extensible** without breaking history
- **Easy to normalize** from bench-centric formats

Practical advice (don’t skip this)

If you’re actually building something:

- **Always store set-centric internally**
- Generate bench-centric views only for UI
- Treat `exercise_id` as optional early on
- Never encode logic in names (e.g. “Bench Press (Drop Set)”)

If you want, I can:

- Add **velocity-based training**
- Add **failure / partial reps**
- Design a **SQL schema** from this
- Show **lossless conversion** between the two models

Just tell me where you’re headed.